

I PROGETTI

Progetto 1 - Appuntamenti

Cosa costruisci. Una lista di persone con un bottone per rimuoverle individualmente e uno per svuotare tutto.

Concetti applicati: useState con array, map, key, eventi, immutabilità.

```
import { useState } from 'react';
import data from './data';

const App = () => {
  const [people, setPeople] = useState(data);

  const rimuoviSingolo = (id) => {
    setPeople(people.filter((persona) => persona.id !== id));
  };

  return (
    <section>
      <div className="container">
        <h2>I prossimi incontri</h2>
        <div className="people-list">
          {people.map(({ id, name, date, img }) => (
            <article key={id} className="person">
              <img src={img} alt={name} />
              <div>
                <h4>{name}</h4>
                <p>{date}</p>
              </div>
              <button onClick={() => rimuoviSingolo(id)}>X</button>
            </article>
          ))}
        </div>
      </div>
    </section>
  )
}
```

```

    </div>

    <button onClick={() => setPeople([])}>Cancella tutto</button>

  </div>

</section>

);
};

```

Il dettaglio tecnico da notare — lo stile inline con le variabili CSS, spiegato nella trascrizione:

```
<h2 style={{ fontSize: "2rem", color: "var(--bg-blue)" }}>
```

Tre cose insieme: la prima coppia di graffe entra in JavaScript, la seconda è l'oggetto stile, `fontSize` diventa `fontSize` in camelCase, e `var(--bg-blue)` dimostra che **le variabili CSS funzionano anche negli stili inline di React**. È un ponte utile fra il CSS del progetto e i valori dinamici.

Perché questo progetto è il primo. Non c'è niente di nuovo rispetto alla Lezione 3, ed è voluto: serve a consolidare il meccanismo `stato → map → evento → nuovo stato → nuovo render` prima di aggiungere complessità.

Progetto 2 - Dark Mode

Cosa costruisci. Un interruttore chiaro/scuro che **ricorda la scelta** dopo il refresh della pagina.

Concetti applicati: `useState` booleano, `useEffect` con dipendenza, manipolazione di classi sul `document`, `localStorage`.

La funzione di toggle

```

const [dark, setDark] = useState(false);

const cambiaModalita = () => {
  setDark((prev) => !prev);
};

```

Applicare il tema al documento

Il tema non riguarda un singolo componente ma tutta la pagina, quindi la classe va sul tag `<html>`. È un side effect, e va in `useEffect`:

```

useEffect(() => {
  const body = document.querySelector("html");
  if (dark) {
    body.classList.add("dark-theme");
  }
}, [dark]);

```

```
} else {  
  body.classList.remove("dark-theme");  
}  
}, [dark]);
```

Il CSS fornito definisce le variabili due volte:

```
:root { --bg-primary: #fff; --text-primary: #333; }  
.dark-theme { --bg-primary: #1a1a1a; --text-primary: #f0f0f0; }
```

Cambiando una classe cambia l'intero schema di colori senza toccare un solo componente.

localStorage — la persistenza

Definizione. Il *localStorage* è uno spazio di memoria del browser, specifico per ogni sito, che sopravvive alla chiusura della scheda e del browser. Salva **solo stringhe**, in coppie chiave-valore.

```
localStorage.setItem('theme', 'dark-theme'); // salva  
localStorage.getItem('theme'); // legge  
localStorage.removeItem('theme'); // cancella
```

Lo ispezioni dai DevTools: scheda **Application** → **Local Storage**.

Passo 1 — salvare ad ogni cambio, dentro lo stesso `useEffect`:

```
useEffect(() => {  
  const body = document.querySelector("html");  
  if (dark) {  
    body.classList.add("dark-theme");  
    localStorage.setItem('theme', 'dark');  
  } else {  
    body.classList.remove("dark-theme");  
    localStorage.setItem('theme', 'light');  
  }  
}, [dark]);
```

Passo 2 — leggere all'avvio. Qui sta il punto interessante: lo stato iniziale non è un valore fisso, ma il risultato di una funzione.

```
const getValueFromLocalStorage = () => {  
  const valore = localStorage.getItem('theme');  
  if (valore) {  
    return valore === 'dark';  
  }  
}
```

```
}  
  
return false;  
  
};
```

```
const [dark, setDark] = useState(getValueFromLocalStorage);
```

Nota importante sulla sintassi: si passa `getValueFromLocalStorage` senza parentesi, non `getValueFromLocalStorage()`. Questo si chiama **lazy initial state**: React esegue la funzione solo al primo render. Con le parentesi, la funzione verrebbe chiamata ad **ogni** render — inutile e costoso, perché il valore iniziale viene ignorato dopo il primo giro.

Estensione realistica: invece di `false`, si può leggere la preferenza di sistema dell'utente:

```
return window.matchMedia('(prefers-color-scheme: dark)').matches;
```

Progetto 3 - Vacanze

Cosa costruisci. Un'app che scarica un elenco di festività da un'API e permette di scorrerle una alla volta.

Concetti applicati: `useEffect` con array vuoto, `async/await`, i tre stati del fetching, render condizionale, componenti riutilizzabili.

Il Title Component — il pattern del componente riutilizzabile

```
const Title = ({ title, subtitle }) => (  
  <div className="section-title">  
    <h2>{title}</h2>  
    {subtitle && <h4>{subtitle}</h4>}  
  </div>  
);
```

`! subtitle &&` è la short circuit evaluation: se il sottotitolo non viene passato, l'`<h4>` semplicemente non esiste. È il modo per rendere una prop opzionale.

Data fetching

```
const App = () => {  
  const [holidays, setHolidays] = useState([]);  
  const [loading, setLoading] = useState(true);  
  const [index, setIndex] = useState(0);  
  
  const fetchHolidays = async () => {
```

```
try {
  const response = await fetch(url);
  const data = await response.json();
  setHolidays(data);
} catch (error) {
  console.error(error);
} finally {
  setLoading(false);
}
};

useEffect(() => {
  fetchHolidays();
}, []);
```

Render condizionale a più livelli

```
if (loading) {
  return <div className="loading"></div>;
}

if (holidays.length === 0) {
  return <h2>Nessuna festività trovata</h2>;
}

return (
  <section>
    <SingleHoliday {...holidays[index]} />
    <button onClick={prevHoliday}>Precedente</button>
    <button onClick={nextHoliday}>Successiva</button>
  </section>
);
};
```

Tre schermate diverse dallo stesso componente: caricamento, lista vuota, contenuto. Nella trascrizione il caricamento è uno spinner CSS (`<div className="loading">`), non un testo — piccolo dettaglio, ma è il modo professionale di farlo.

Next e Prev — la navigazione circolare

```
const nextHoliday = () => {
  setIndex((prev) => {
    const nuovo = prev + 1;
    return nuovo > holidays.length - 1 ? 0 : nuovo;
  });
};

const prevHoliday = () => {
  setIndex((prev) => {
    const nuovo = prev - 1;
    return nuovo < 0 ? holidays.length - 1 : nuovo;
  });
};
```

L'idea: quando superi l'ultimo elemento torni al primo, quando scendi sotto zero salti all'ultimo. Lo stato non contiene il dato, contiene **l'indice** del dato. È un pattern che ritroverai identico nel progetto Slider.

Progetto 4 - Slider

Cosa costruisci. Un carosello di recensioni con frecce, valutazione a stelle e scorrimento automatico.

Concetti applicati: classi CSS dinamiche, `useEffect` con dipendenze multiple, `setInterval` e `cleanup`, generazione di elementi con `Array.from`.

Traslare le slide — il meccanismo

L'idea, spiegata bene nella trascrizione. Tutte le slide sono sovrapposte nello stesso punto. Tre classi CSS decidono la posizione:

```
.slide { position: absolute; transition: all 0.3s linear; }
.activeSlide { transform: translateX(0); opacity: 1; }
.nextSlide { transform: translateX(100%); opacity: 0; }
.lastSlide { transform: translateX(-100%); opacity: 0; }
```

Il compito di React è **assegnare la classe giusta a ogni slide** in base alla posizione rispetto all'indice attivo:

```
const [active, setActive] = useState(0);

{recensioni.map((recensione, indiceSlide) => {
  let position = "nextSlide";
  if (indiceSlide === active) {
    position = "activeSlide";
  }
  if (indiceSlide === active - 1 || (active === 0 && indiceSlide === recensioni.length - 1)) {
    position = "lastSlide";
  }
  return <Slide key={recensione.id} {...recensione} position={position} />;
})}
```

La seconda condizione gestisce il caso limite: quando sei sulla prima slide (`active === 0`), la "precedente" è l'ultima dell'array.

Nel componente:

```
const Slide = ({ nome, testo, stelle, position }) => (
  <article className={`slide ${position}`} >
    { /* ... */ }
  </article>
);
```

Il template literal compone la classe base con quella dinamica. **Nessuna manipolazione diretta del DOM**: si cambia solo lo stato, il CSS fa il resto. È il modo React di fare animazioni.

La funzione per creare le stelle

```
const Stelle = ({ numero }) => {
  const stelle = Array.from({ length: numero }, (_, index) => (
    <FaStar key={index} />
  ));
  return <div className="stelle">{stelle}</div>;
};
```

`Array.from({ length: 4 })` crea un array di 4 posizioni vuote su cui iterare. Qui la `key` con l'indice è accettabile: le stelle sono identiche, non riordinabili e senza stato proprio.

Auto Slider

```
useEffect(() => {
```

```
const slider = setInterval(() => {
  setActive((prev) => (prev + 1) % recensioni.length);
}, 3000);

return () => clearInterval(slider);
}, [active]);
```

Tre cose da capire qui:

1. **clearInterval nella cleanup** è obbligatorio. Senza, ogni volta che **active** cambia si crea un nuovo interval senza cancellare il vecchio: dopo dieci click hai dieci timer sovrapposti e lo slider impazzisce.
2. **[active] nelle dipendenze** fa ripartire il conto alla rovescia quando l'utente clicca manualmente. Senza, lo slider potrebbe avanzare da solo mezzo secondo dopo il click dell'utente — comportamento sgradevole.
3. La **forma funzionale setActive(prev => ...)** è indispensabile. Con `setActive(active + 1)`, il callback dell'interval leggerebbe sempre il valore di **active** congelato al momento in cui l'interval è stato creato. È la *stale closure* della Lezione 3, e qui si vede il danno concreto.

L'operatore % (modulo) è la scorciatoia elegante per il ciclo circolare: $(3 + 1) \% 4 = 0$.

Progetto 5 – Gelateria

Cosa costruisci. Un menu di prodotti filtrabile per categoria, con i dati presi da un'API.

Concetti applicati: stati derivati, filtri dinamici, **Set** per l'unicità, due stati per la stessa collezione.

Il Menu Component — categorie generate dai dati

Il problema. Le categorie non vanno scritte a mano: vanno estratte dai prodotti. Se domani l'API aggiunge una categoria, il menu deve aggiornarsi da solo.

```
const categorie = ["tutti", ...new Set(prodotti.map((p) => p.categoria))];
```

Scomposto:

1. `prodotti.map(p => p.categoria) → ["gelati", "granite", "gelati", "coppe", "granite"]`
2. `new Set(...)` → rimuove i duplicati (un **Set** è una collezione di valori unici)
3. `[...]` → riconverte il **Set** in array, perché **Set** non ha `map`
4. `"tutti"` in testa come opzione predefinita

Risultato: `["tutti", "gelati", "granite", "coppe"]`.

```
const Menu = ({ categorie, filtra }) => (
  <div className="menu">
```

```

    {categorie.map((cat, i) => (
      <button key={i} onClick={() => filtra(cat)}>{cat}</button>
    )))}
  </div>
);

```

La funzione di filtro — due stati per la stessa collezione

Il concetto chiave di questo progetto. Servono **due** stati:

```

const [tuttiProdotti, setTuttiProdotti] = useState([]); // originale, mai modificato
const [prodotti, setProdotti] = useState([]);           // visualizzato, filtrato

const filtra = (categoria) => {
  if (categoria === "tutti") {
    setProdotti(tuttiProdotti);
    return;
  }

  const filtrati = tuttiProdotti.filter((p) => p.categoria === categoria);
  setProdotti(filtrati);
};

```

Perché due. Se filtrassi direttamente `prodotti`, dopo il primo filtro avresti perso gli altri elementi: cliccando "tutti" non tornerebbe più nulla, perché l'array originale non esiste più. `tuttiProdotti` è la fonte immutabile; `prodotti` è la vista corrente.

Al fetching si popolano entrambi:

```

const fetchProdotti = async () => {
  const response = await fetch(url);
  const data = await response.json();
  setTuttiProdotti(data);
  setProdotti(data);
  setLoading(false);
};

```

Alternativa moderna, che vale la pena conoscere. Invece di duplicare lo stato, si tiene solo la categoria selezionata e si calcola la lista filtrata durante il render:

```

const [categoria, setCategoria] = useState("tutti");

```

```
const prodottiFiltrati = categoria === "tutti"
  ? tuttiProdotti
  : tuttiProdotti.filter((p) => p.categoria === categoria);
```

Meno stato da sincronizzare, meno possibilità di bug. È il principio: *non mettere nello stato ciò che puoi derivare*.

Nota: il corso ha una lezione marcata "UpDate – API (importante)". Le API pubbliche cambiano o muoiono; se nel repository trovi un URL non funzionante, quella lezione contiene l'endpoint aggiornato.

Progetto 6 – Color-Grading

Cosa costruisci. Inserisci un colore esadecimale e l'app genera una scala di 20 sfumature, cliccabili per copiare il valore.

Concetti applicati: form controllato, libreria esterna, `navigator.clipboard`, gestione errori nell'input, `useEffect` con timer.

Form Input

```
const [color, setColor] = useState("#f15025");
const [error, setError] = useState(false);
const [list, setList] = useState(new Values("#f15025").all(10));

const handleSubmit = (e) => {
  e.preventDefault();
  try {
    const colori = new Values(color).all(10);
    setList(colori);
    setError(false);
  } catch (err) {
    setError(true);
  }
};
```

```
<form onSubmit={handleSubmit}>
  <input
    type="text"
```

```

value={color}

onChange={(e) => setColor(e.target.value)}

className={error ? "error" : ""}

/>

<button type="submit" style={{ background: color }}>invia</button>

</form>

```

Due dettagli: il bottone usa il colore inserito come sfondo (feedback immediato), e la classe `error` sull'input segnala visivamente il valore non valido.

La libreria `values.js`

```
npm install values.js
```

Definizione. È una libreria che, dato un colore, genera automaticamente tinte (*tints*, più chiare) e ombre (*shades*, più scure).

```

import Values from 'values.js';

new Values("#f15025").all(10);

// ritorna un array di ~20 oggetti:

// [{ rgb: [r,g,b], weight: 100, type: 'shade', hex: '...' }, ...]

```

La lezione di metodo: non si reimplementa a mano la conversione HSL/RGB. Si cerca una libreria affidabile, si legge la documentazione, si integra. È esattamente quello che fai nel lavoro reale.

Single Color Component e copia negli appunti

```

const SingleColor = ({ rgb, weight, index, hexColor }) => {
  const [alert, setAlert] = useState(false);
  const bcg = rgb.join(",");
  const hex = `${hexColor}`;

  useEffect(() => {
    const timeout = setTimeout(() => setAlert(false), 3000);
    return () => clearTimeout(timeout);
  }, [alert]);

  return (
    <article
      className = {`color ${index > 10 && "color-light"}`}
      style = {{ backgroundColor: `rgb(${bcg})` }}
    >

```

```

onClick={ () => {
    navigator.clipboard.writeText(hex);
    setAlert(true);
  }}
>
<p>{weight}%</p>
<p>{hex}</p>
{alert && <p className="alert">copla negli appunti</p>}
</article>
); };

```

Tre cose da notare:

1. **rgb.join(",")** trasforma `[241, 80, 37]` in `"241,80,37"`, che è ciò che serve dentro `rgb(...)`.
2. **navigator.clipboard.writeText()** è l'API nativa del browser per copiare. Funziona solo su HTTPS o localhost — è una restrizione di sicurezza, non un bug.
3. **L>alert che sparisce da solo.** Il `setTimeout` con `cleanup` nel `useEffect` è il pattern per i messaggi temporanei. `clearTimeout` nella `cleanup` impedisce che timer accumulati facciano sparire l>alert prima del tempo su click ripetuti.

Nota anche che **ogni SingleColor ha il suo stato alert**: cliccando un colore, solo quello mostra il messaggio. È la conseguenza diretta del fatto che ogni istanza di un componente ha stato indipendente.

Gestione errore input

L'approccio col `try/catch` è quello giusto: la libreria lancia un'eccezione sui valori non validi, e tu la intercetti invece di validare a mano con una regex.

PROGETTO 7 — Navbar

Cosa costruisci. Una barra di navigazione responsive che si apre e chiude con animazione fluida.

Concetti applicati: `useRef` su due elementi, `useEffect` per misurare il DOM, altezza dinamica.

Il problema dell'animazione CSS

Perché non basta il CSS. Per animare un'apertura serve una transizione su `height`. Ma `height: auto` **non è animabile**: il browser non sa da che valore partire. Serve un'altezza in pixel, e quell'altezza dipende dal numero di voci di menu — quindi va calcolata a runtime.

La soluzione con due useRef

```
import { useState, useRef, useEffect } from 'react';

const Navbar = () => {
  const [show, setShow] = useState(false);
  const linksContainerRef = useRef(null); // il contenitore con altezza controllata
  const linksRef = useRef(null);        // la lista, per misurarne l'altezza reale

  useEffect(() => {
    const linksHeight = linksRef.current.getBoundingClientRect().height;
    if (show) {
      linksContainerRef.current.style.height = `${linksHeight}px`;
    } else {
      linksContainerRef.current.style.height = '0px';
    }
  }, [show]);

  return (
    <nav>
      <div className="nav-header">
        <img src={logo} alt="logo" />
        <button onClick={() => setShow(!show)}>
          <FaBars />
        </button>
      </div>

      <div className="links-container" ref={linksContainerRef}>
        <ul className="links" ref={linksRef}>
          {links.map(({ id, url, text }) => (
            <li key={id}><a href={url}>{text}</a></li>
          ))}
        </ul>
      </div>
    </nav>
  );
}
```

```
</nav>

);

};
```

Il meccanismo, passo per passo:

1. `.links-container` ha `overflow: hidden` e `transition: height 0.3s`.
2. `.links` (la lista dentro) ha la sua altezza naturale, qualunque sia.
3. `getBoundingClientRect().height` **misura** quell'altezza reale in pixel.
4. Quell'altezza viene applicata al contenitore quando il menu è aperto, `0px` quando è chiuso.
5. Il CSS anima la transizione fra i due valori.

Perché servono due ref. Uno per **misurare** (l'elemento con l'altezza naturale), uno per **modificare** (il contenitore che deve animarsi). Se ne usassi uno solo, l'atto di impostare l'altezza a `0px` cancellerebbe la misura.

Perché useRef e non useState. Perché non stiamo conservando un dato da mostrare: stiamo leggendo e scrivendo direttamente sul DOM. Una misura in pixel non ha bisogno di causare un render.

Il vantaggio rispetto al display: none. Il menu funziona a qualsiasi numero di voci, senza altezze fisse scritte nel CSS, e con animazione fluida. È un caso in cui `useRef` è la soluzione giusta e non un ripiego.

PROGETTO 8 — Sidebar

Cosa costruisci. Una sidebar laterale e un modal, entrambi controllati da uno stato globale.

Concetti applicati: `useContext` in un caso reale, custom hook per il contesto, **styled-components**.

Implementazione del Context

```
// context.js

import { createContext, useContext, useState } from 'react';

const AppContext = createContext();

export const AppProvider = ({ children }) => {
  const [isSidebarOpen, setIsSidebarOpen] = useState(false);
  const [isModalOpen, setIsModalOpen] = useState(false);

  const openSidebar = () => setIsSidebarOpen(true);
```

```

const closeSidebar = () => setIsSidebarOpen(false);
const openModal = () => setIsModalOpen(true);
const closeModal = () => setIsModalOpen(false);

return (
  <AppContext.Provider value={{
    isSidebarOpen, isModalOpen,
    openSidebar, closeSidebar, openModal, closeModal
  }}>
    {children}
  </AppContext.Provider>
);
};

```

export const useGlobalContext = () => useContext(AppContext);

E in `index.js`:

```

<AppProvider>
  <App />
</AppProvider>

```

L'uso nei componenti

```

const Navbar = () => {
  const { openSidebar } = useGlobalContext();
  return <button onClick={openSidebar}><FaBars /></button>;
};

const Sidebar = () => {
  const { isSidebarOpen, closeSidebar } = useGlobalContext();
  return (
    <aside className={`sidebar ${isSidebarOpen ? 'show-sidebar' : ''}`>
      <button onClick={closeSidebar}><FaTimes /></button>
      { /* links */ }
    </aside>
  );
};

```

```
);  
};
```

Il valore concreto. La Navbar apre la sidebar, la Sidebar si chiude da sola, il Modal è controllato da un terzo componente. Nessuno dei tre è figlio dell'altro, eppure condividono lo stato senza una sola prop passata. Senza Context, dovresti tenere gli stati in App e passarli a tutti — il prop drilling della Lezione 3.

Funzioni separate invece di un toggle. `openSidebar` e `closeSidebar` invece di `toggleSidebar` è una scelta consapevole: rende esplicito cosa fa ogni bottone. Anche la trascrizione tocca questo punto a proposito della leggibilità dei nomi di funzione.

Styled Components

```
npm install styled-components
```

Definizione. Libreria che permette di scrivere CSS dentro JavaScript, generando componenti già stilizzati.

```
import styled from 'styled-components';  
  
const Aside = styled.aside`  
  position: fixed;  
  width: 300px;  
  height: 100%;  
  background: var(--clr-white);  
  transform: translateX(-100%);  
  transition: all 0.3s ease-in-out;  
  
  ${props => props.isOpen && `  
    transform: translateX(0);  
  `}  
  
  @media screen and (min-width: 768px) {  
    width: 400px;  
  }  
`;  
  
// uso
```

```
<Aside isOpen={isSidebarOpen}>...</Aside>
```

I vantaggi:

- Lo stile è **scoped**: le classi generate sono uniche, niente collisioni di nomi.
- Lo stile può dipendere dalle props, con JavaScript vero.
- CSS e componente stanno nello stesso file: cancelli il componente, cancelli il suo stile.

Gli svantaggi: il CSS viene calcolato a runtime (costo sulle performance), e la sintassi con i template literal è meno familiare. Oggi molti progetti preferiscono **Tailwind CSS** o **CSS Modules**, che hanno costo zero a runtime. Vale la pena conoscere styled-components perché la Lezione 5 ci costruisce sopra tutto il design system con **styled-system**.

PROGETTO 9 — Cart

Cosa costruisci. Un carrello e-commerce completo: quantità, rimozione, totale, contatore.

Concetti applicati: `useReducer` + `useContext` insieme, gestione di stato complesso, calcoli derivati. È il progetto più importante della lezione, perché è il modello mentale che userai in Redux.

Lo stato iniziale

```
const initialState = {  
  loading: true,  
  products: [],  
  error: false,  
  total: 0,  
  itemCounter: 0  
};
```

Cinque valori correlati. Gestirli con cinque `useState` separati significherebbe aggiornarli a mano in ogni funzione, con il rischio di dimenticarne uno.

Il reducer

```
// reducer.js  
  
const reducer = (state, action) => {  
  switch (action.type) {  
    case 'FETCH_START':  
      return { ...state, loading: true };  
  
    case 'FETCH_SUCCESS':  
      return { ...state, loading: false, products: action.payload };  
  }  
}
```

case 'FETCH_ERROR':

```
return { ...state, loading: false, error: true };
```

case 'REMOVE_ITEM':

```
return {  
  ...state,  
  products: state.products.filter((p) => p.id !== action.payload)  
};
```

case 'INCREASE':

```
return {  
  ...state,  
  products: state.products.map((p) =>  
    p.id === action.payload ? { ...p, amount: p.amount + 1 } : p  
  )  
};
```

case 'DECREASE':

```
return {  
  ...state,  
  products: state.products  
    .map((p) =>  
      p.id === action.payload ? { ...p, amount: p.amount - 1 } : p  
    )  
    .filter((p) => p.amount > 0)    // rimuove se arriva a zero  
};
```

case 'GET_TOTALS': {

```
const { total, itemCounter } = state.products.reduce(  
  (acc, prodotto) => {  
    acc.total += prodotto.price * prodotto.amount;
```

```

    acc.itemCounter += prodotto.amount;

    return acc;
  },
  { total: 0, itemCounter: 0 }
);
return { ...state, total: parseFloat(total.toFixed(2)), itemCounter };
}

case 'CLEAR_CART':
  return { ...state, products: [] };

default:
  throw new Error(`Nessuna azione corrispondente a: ${action.type}`);
}
};

```

Tre tecniche da assimilare qui:

- 1. map con ternario per modificare un solo elemento.** Non si cerca l'indice e si modifica in posto: si mappa l'intero array ritornando l'oggetto originale per tutti tranne quello che ci interessa, di cui si crea una copia modificata. Immutabilità rispettata a ogni livello.
- 2. map + filter concatenati** in DECREASE: prima si decrementa, poi si eliminano gli elementi arrivati a zero. Una sola espressione, leggibile.
- 3. reduce per i totali.** Un unico passaggio sull'array calcola sia il totale in euro sia il numero di pezzi. `toFixed(2)` arrotonda ai centesimi (e `parseFloat` lo riconverte a numero, perché `toFixed` ritorna una stringa — classico inciampo con i prezzi).

Il calcolo automatico dei totali

```

useEffect(() => {
  dispatch({ type: 'GET_TOTALS' });
}, [state.products]);

```

Ogni volta che i prodotti cambiano — aggiunta, rimozione, cambio quantità — i totali si ricalcolano da soli. Non devi ricordarti di chiamarli in ogni funzione. È il vantaggio di ragionare in termini di *reazione al cambiamento* invece che di *sequenza di istruzioni*.

Il Context che espone tutto

```

export const AppProvider = ({ children }) => {
  const [state, dispatch] = useReducer(reducer, initialState);

```

```

const fetchData = async () => {
  dispatch({ type: 'FETCH_START' });
  try {
    const response = await fetch(url);
    const data = await response.json();
    dispatch({ type: 'FETCH_SUCCESS', payload: data });
  } catch (error) {
    dispatch({ type: 'FETCH_ERROR' });
  }
};

useEffect(() => { fetchData(); }, []);

return (
  <AppContext.Provider value={{
    ...state,
    removeItem: (id) => dispatch({ type: 'REMOVE_ITEM', payload: id }),
    increase: (id) => dispatch({ type: 'INCREASE', payload: id }),
    decrease: (id) => dispatch({ type: 'DECREASE', payload: id }),
    clearCart: () => dispatch({ type: 'CLEAR_CART' })
  }}>
    {children}
  </AppContext.Provider>
);
};

```

Il pattern completo: i componenti **non conoscono il reducer né le action type**. Chiamano `increase(id)` e basta. La logica è incapsulata nel provider, l'interfaccia è pulita. È esattamente l'architettura che Redux Toolkit formalizza.

E nel componente:

```

const CartItem = ({ id, img, title, price, amount }) => {
  const { removeItem, increase, decrease } = useGlobalContext();

```

```

return (
  <article className="cart-item">
    <img src={img} alt={title} />
    <div>
      <h4>{title}</h4>
      <h4>€{price}</h4>
      <button onClick={() => removeItem(id)}>rimuovi</button>
    </div>
    <div>
      <button onClick={() => increase(id)}><FaChevronUp /></button>
      <p>{amount}</p>
      <button onClick={() => decrease(id)}><FaChevronDown /></button>
    </div>
  </article>
);
};

```

PROGETTO 10 — Cocktail (Wiki Drink)

Cosa costruisci. Un sito multi-pagina completo per cercare cocktail: home con ricerca, pagina di dettaglio, about, contatti, sidebar, footer, pagina 404, e deploy finale.

È il progetto conclusivo: 25 lezioni che mettono insieme tutto. Compare anche nella Lezione 3 come "Progetto 10: Wiki Drink" a proposito dell'ottimizzazione.

Struttura del progetto

```

src/
├── components/  Navbar, Footer, Sidebar, Hero, Cocktail, Form, Card
├── screens/     Home, About, Contact, SingleCocktail, ErrorPage
├── hooks/       useFetch, useTitle
├── context.js
└── App.js

```

La separazione `components` / `screens` è quella introdotta con React Router: le screen sono pagine associate a una rotta, i components sono pezzi riutilizzabili.

Il Router

```
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';
```

```
const App = () => (  
  <Router>  
    <Navbar />  
    <Sidebar />  
    <Routes>  
      <Route path="/" element={<Home />} />  
      <Route path="/about" element={<About />} />  
      <Route path="/contact" element={<Contact />} />  
      <Route path="/cocktail/:id" element={<SingleCocktail />} />  
      <Route path="*" element={<ErrorPage />} />  
    </Routes>  
    <Footer />  
  </Router>  
)
```

Navbar, Sidebar e Footer stanno **fuori da Routes**: restano fissi mentre cambia solo il contenuto centrale.

Nota: la trascrizione ha una lezione dedicata "UpDate: React Router v6 pt1", perché il progetto era stato girato con la v5 e poi aggiornato. Segui la versione v6.

useFetch — il custom hook

```
// hooks/useFetch.js  
  
const useFetch = (url) => {  
  const [data, setData] = useState(null);  
  const [loading, setLoading] = useState(true);  
  const [error, setError] = useState(null);  
  
  useEffect(() => {  
    const fetchData = async () => {  
      setLoading(true);  
      try {
```

```

    const response = await fetch(url);

    const result = await response.json();

    setData(result.drinks);

    setError(null);

  } catch (err) {

    setError(err.message);

  } finally {

    setLoading(false);

  }

};

fetchData();

}, [url]);

return { data, loading, error };

};

```

Usato sia nella Home (ricerca) sia nella pagina di dettaglio. `[url]` nelle dipendenze fa scattare una nuova chiamata ogni volta che l'URL cambia — cioè ad ogni ricerca.

`useTitle` — il secondo custom hook

```

// hooks/useTitle.js

const useTitle = (title) => {

  useEffect(() => {

    document.title = `${title} | Wiki Drink;

  }, [title]);

};

```

Tre righe, e ogni pagina ha il titolo giusto nella scheda del browser:

```

const About = () => {

  useTitle("About");

  return <section>...</section>;

};

```

È l'esempio minimo che fa capire il valore dei custom hook: incapsulare un comportamento ripetitivo in una riga riutilizzabile.

Single Cocktail Page — useParams in azione

```
const SingleCocktail = () => {
  const { id } = useParams();
  const { data, loading } = useFetch(`${baseUrl}/lookup.php?i=${id}`);

  if (loading) return <Loading />;
  if (!data) return <h2>Nessun cocktail trovato</h2>;

  const cocktail = data[0];
  useTitle(cocktail.strDrink);

  return (
    <section>
      <h2>{cocktail.strDrink}</h2>
      <img src={cocktail.strDrinkThumb} alt={cocktail.strDrink} />
      <p>{cocktail.strInstructions}</p>
    </section>
  );
};
```

L'id dall'URL costruisce l'URL dell'API. Un solo componente serve tutti i cocktail esistenti.

(Attenzione: nel codice sopra `useTitle` sta dopo i return condizionali — violazione della Regola 1 degli hook. Nel progetto reale va spostato prima dei return o dentro un `useEffect` che gestisce il caso `null`.)

Scroll Position Function

Il problema descritto nella trascrizione. L'utente cerca "a", ottiene 25 risultati, scorre in fondo, clicca il diciottesimo. Guarda la scheda, torna indietro — e si ritrova in cima alla lista, dovendo riscorrere tutto. Con tre risultati è tollerabile; con venticinque è irritante.

La soluzione: salvare la posizione di scroll nel Context prima di navigare, e ripristinarla al ritorno.

// nel Context

```
const [scrollPosition, setScrollPosition] = useState(0);
const savePosition = () => setScrollPosition(window.scrollY);
```

```
// nel Cocktail component — si usa useNavigate, non Link
const Cocktail = ({ id, name, image }) => {
  const { savePosition } = useGlobalContext();
  const navigate = useNavigate();

  const handleClick = () => {
    savePosition();          // prima salvo
    navigate(`/cocktail/${id}`); // poi navigo
  };

  return <div onClick={handleClick}>...</div>;
};
```

```
// nella Home — ripristino al montaggio
useEffect(() => {
  window.scrollTo(0, scrollPosition);
}, []);
```

Perché useNavigate invece di Link. La trascrizione lo spiega esplicitamente: serve un elemento cliccabile che **prima esegua una funzione e poi navighi**. Un `Link` naviga e basta. È il caso d'uso canonico della navigazione programmatica.

Deploy su Netlify

Vale quanto visto nella Lezione 2, con due accortezze specifiche:

Il file `_redirects` in `public/`, obbligatorio con React Router:

```
/* /index.html 200
```

Senza, ricaricando la pagina su `/about` ottieni un 404.

Le variabili d'ambiente, se l'API richiede una chiave: vanno reinserite nelle impostazioni Netlify, perché il file `.env` non viene caricato su Git.

=====

Cosa studiare dopo React

L'ultima lezione del file suggerisce i passi successivi. Il corso spinge in particolare su **Gatsby**, descritto come un generatore di siti statici basato su React e GraphQL, con code splitting, server-side rendering e ottimizzazione automatica delle immagini.

Una precisazione onesta: nel 2026 Gatsby è molto meno rilevante di quanto fosse quando il corso è stato registrato. I percorsi più sensati oggi sono:

- **Next.js** — il framework React di riferimento: routing, rendering server-side, API routes. È lo standard di fatto per le applicazioni React in produzione.
- **TypeScript** — sostituisce le PropTypes con un sistema di tipi serio. Praticamente richiesto in ambito professionale.
- **TanStack Query** (ex React Query) — gestisce data fetching, cache, sincronizzazione. Rende obsoleto il pattern `useEffect` + `useState` per i dati remoti.
- **Testing** — Vitest e React Testing Library.

=====